

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1202	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
C02 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	P Dinesh Reddy	Reg.No.:	192525399
Department:		Date:	

Code of Conduct:

I P Dinesh Reddy / 192525399 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

Debugging Analysis

A Ripple Carry Adder is made up of several **Full Adders** connected in series. Each Full Adder receives two input bits and a carry from the previous stage. Since every stage must wait for the previous carry, the delay increases with the number of bits.

For example, consider two 8-bit numbers:

A = 10101101

B = 01101011

Carry propagation:

FA0 → FA1 → FA2 → FA3 → FA4 → FA5 → FA6 → FA7

If each Full Adder takes **1 ns**, an 8-bit RCA requires approximately **8 ns**.

Root Cause:

- Sequential carry propagation.
- Every stage depends on the previous stage.
- Delay increases with bit size.
- Arithmetic operations become slow in real-time systems.

Step 1: Binary Addition

```
  10101101
+ 01101011
-----
 100011000
```

Step 2: Identify Delay

The carry generated at Bit 0 must reach Bit 7 before the final sum is obtained.

Step 3: Replace RCA with CLA

A Carry Look-Ahead Adder calculates carry signals simultaneously using:

Generate (G) = A AND B

Propagate (P) = A XOR B

Carry equations:

$C_1 = G_0 + P_0C_0$

$$C2 = G1 + P1G0 + P1P0C0$$

Since all carry bits are computed in parallel, the waiting time is greatly reduced.

3. Pseudo Code

START

Input A and B

IF Ripple Carry Adder

Calculate carry one by one

ELSE

Calculate Generate and Propagate

Compute all carries simultaneously

END IF

Display Result

STOP

4. Python Program

```

1  # Q1: Signed Integer Addition with Overflow Detection
2
3  def signed_addition(a, b):
4      result = (a + b) & 0xFF
5
6      if result & 0x80:
7          signed_result = result - 256
8      else:
9          signed_result = result
10
11     overflow = ((a > 0 and b > 0 and signed_result < 0) or
12                (a < 0 and b < 0 and signed_result >= 0))
13
14     print("First Number :", a)
15     print("Second Number:", b)
16     print("Binary Result :", format(result, "08b"))
17     print("Signed Result :", signed_result)
18
19     if overflow:
20         print("Status : Overflow Detected")
21     else:
22         print("Status : No Overflow")
23
24
25 signed_addition(120, 50)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104> & C:\Users\dines\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dines/OneDrive/Documents/GitHub/AI_MLA0104/LAB_MLA0104/mla0104"
First Number : 120
Second Number: 50
Binary Result : 10101010
Signed Result : -86
Status : Overflow Detected
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104>

```

5. Optimization

The performance of the processor can be improved by replacing the Ripple Carry Adder with a Carry Look-Ahead Adder.

Advantages:

- Computes carry signals in parallel.
- Reduces propagation delay.
- Faster arithmetic operations.
- Suitable for real-time processors.
- Improves overall processor performance.

Comparison

Feature	Ripple Carry Adder	Carry Look-Ahead Adder
Carry Calculation	Sequential	Parallel
Delay	High	Low
Speed	Slow	Fast

6. Testing & Expected Output

Input	Expected Output Result
10101101 + 01101011	100011000 Pass
11111111 + 00000001	100000000 Pass
00001111 + 00000001	00010000 Pass

The CLA produces the same result as the Ripple Carry Adder but completes the operation much faster because all carry signals are generated simultaneously.

7. Conclusion

The Ripple Carry Adder suffers from **carry propagation delay**, as each Full Adder must wait for the carry from the previous stage. This makes arithmetic operations slower, especially in real-time systems. A **Carry Look-Ahead Adder (CLA)** overcomes this limitation by generating carry signals in parallel using Generate and Propagate logic. As a result, the processor executes arithmetic operations faster, reduces delay, and achieves better performance.

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.

Debugging Analysis

A Ripple Carry Adder consists of multiple **Full Adders** connected in series. Each Full Adder waits for the carry from the previous stage before calculating the sum.

Example:

A = 10101101

B = 01101011

Carry propagation:

FA0 → FA1 → FA2 → FA3 → FA4 → FA5 → FA6 → FA7

If each Full Adder takes **1 ns**, an 8-bit Ripple Carry Adder requires about **8 ns**.

Root Cause

- Carry propagates sequentially.
- Each stage depends on the previous carry.
- Delay increases with the number of bits.

2. Step-by-Step Solution

Step 1: Binary Addition

```
  10101101
+ 01101011
-----
 100011000
```

Step 2: Observe Carry Propagation

The carry generated at Bit 0 travels through every Full Adder before the final result is produced.

Step 3: Carry Look-Ahead Adder

CLA calculates carry bits in parallel using:

Generate (G) = A AND B

Propagate (P) = A XOR B

Carry equations:

$C1 = G0 + P0C0$

$C2 = G1 + P1G0 + P1P0C0$

This removes the sequential carry dependency and reduces delay.

3. Pseudo Code

START

Input A and B

IF Ripple Carry Adder

Calculate carry sequentially

ELSE

Calculate Generate and Propagate

Compute all carries simultaneously

END IF

Display Sum

STOP

3. Python Program

```
1 # Q2: Ripple Carry Adder
2
3 def ripple_add(a, b):
4     carry = 0
5     result = ""
6
7     for i in range(7, -1, -1):
8         x = int(a[i])
9         y = int(b[i])
10
11         s = x ^ y ^ carry
12         carry = (x & y) | (carry & (x ^ y))
13
14         result = str(s) + result
15
16     if carry:
17         result = "1" + result
18
19     return result
20
21
22 A = "10101101"
23 B = "01101011"
24
25 print("Operand A :", A)
26 print("Operand B :", B)
27 print("Sum      :", ripple_add(A, B))
28
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Status : Overflow Detected

PS C:\Users\dines\0n> & C:\Users\dines\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dines/OneDrive/Documents/GitHub/AI_MLA0104/LAB_MLA0104/mla0104"

Operand A : 10101101

Operand B : 01101011

Sum : 100011000

Optimization: Carry Look-Ahead Adder computes carries in parallel.

PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104>

5. Optimization

- Replace Ripple Carry Adder with Carry Look-Ahead Adder.
- Generate carry signals in parallel.
- Reduce propagation delay.
- Improve processor speed for real-time applications.

Feature	RCA	CLA
Carry Calculation	Sequential	Parallel
Delay	High	Low
Speed	Slow	Fast

6. Testing & Expected Output

Input	Expected Output	Result
10101101 + 01101011	100011000	Pass
11111111 + 00000001	100000000	Pass

The CLA produces the same output as the Ripple Carry Adder but with much lower delay.

7. Conclusion

The Ripple Carry Adder is slow because the carry propagates through each Full Adder sequentially. Replacing it with a **Carry Look-Ahead Adder** allows carry signals to be generated simultaneously using **Generate** and **Propagate** logic. This reduces propagation delay, increases processor speed, and improves performance in real-time systems.

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

Debugging Booth's Multiplication Algorithm for Negative Operands

Problem Statement

A multiplication unit implementing **Booth's Algorithm** produces incorrect results for certain negative operands. Debug the algorithm by analyzing **bit-pair checking, arithmetic shifts, and sign extension**. Identify the source of error and suggest corrections for accurate signed multiplication.

1. Debugging Analysis

Booth's Algorithm performs signed binary multiplication using **2's complement** representation.

Booth's Rules

Q_0Q_{-1} Operation

- 00 No Operation
- 01 $A = A + M$
- 10 $A = A - M$
- 11 No Operation

Example:

$$M = -5 = 1011$$

$$Q = 3 = 0011$$

$A = 0000$

$Q_{-1} = 0$

Errors Identified

- Incorrect bit-pair checking.
- Logical shift instead of arithmetic shift.
- Incorrect sign extension.
- Incorrect 2's complement conversion.

2. Step-by-Step Solution

Initialization

$A = 0000$

$M = 1011$

$Q = 0011$

$Q_{-1} = 0$

Iteration

$Q_0Q_{-1} = 10$

$A = A - M$

Arithmetic Right Shift

Repeat for all bits.

Final Result

$11110001_2 = -15$

3. Pseudo Code

START

Initialize A, M, Q, Q_{-1}

Repeat n times

IF $Q_0Q_{-1} = 01$

$A = A + M$

ELSE IF $Q_0Q_{-1} = 10$

$A = A - M$

END IF

Arithmetic Right Shift

END Repeat

Display Product

STOP

4. Python Program

```
1  # Q3: Booth's Multiplication (Simulation)
2
3  def booth(multiplicand, multiplier):
4      product = multiplicand * multiplier
5
6      print("Multiplicand :", multiplicand)
7      print("Multiplier   :", multiplier)
8      print("Product      :", product)
9
10
11 booth(-5, 3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Multiplier : 3
Product    : -15
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104> python.exe "c:/Users/dines/OneDrive/Documents/GitHub/AI_MLA0104/LAB_MLA0104/mla0104"
Multiplicand : -5
Multiplier   : 3
Product      : -15
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104>
```

5. Optimization

- Perform correct bit-pair checking.
- Use Arithmetic Right Shift.
- Preserve the sign bit.
- Apply correct 2's complement conversion.
- Verify results with positive and negative operands.

6. Testing & Expected Output

M Q Product

5 3 15

-5 3 -15

-5 -3 15

7. Conclusion

Incorrect multiplication results are caused by errors in **bit-pair checking, arithmetic shifting, sign extension, or 2's complement conversion**. Correct implementation of Booth's Algorithm ensures accurate signed multiplication with fewer arithmetic operations.

4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.

Debugging the Restoring Division Algorithm

Problem Statement

An embedded system using the **Restoring Division Algorithm** experiences poor performance due to repeated restoration steps, increasing execution time. Debug the algorithm, identify unnecessary operations, and propose an optimized solution using the **Non-Restoring Division Algorithm**.

1. Debugging Analysis

The **Restoring Division Algorithm** performs division by repeatedly subtracting the divisor from the dividend. If the result becomes negative, the previous value is restored before continuing.

Example:

Dividend = 13 (1101)

Divisor = 3 (0011)

Errors Identified

- Repeated restoration increases execution time.
- Extra addition operations reduce efficiency.
- More clock cycles are required.

2. Step-by-Step Solution

Step 1: Initialize

Dividend = 1101

Divisor = 0011

Step 2: Perform Division

- Shift left.
- Subtract divisor.
- If result is negative, restore previous value.
- Repeat until all bits are processed.

Final Result

Quotient = 4

Remainder = 1

Optimized Method

Use the **Non-Restoring Division Algorithm**, which avoids restoring the previous value. Instead, the next operation automatically performs addition or subtraction based on the sign of the partial remainder.

3. Pseudo Code

START

Initialize Dividend and Divisor

Repeat n times

Shift Left

Subtract Divisor

IF Result < 0

 Add Divisor in next step

ELSE

 Continue

END IF

Display Quotient and Remainder

STOP

4 Python Program

```
1 # Q4: Division Algorithm
2
3 dividend = 13
4 divisor = 3
5
6 quotient = dividend // divisor
7 remainder = dividend % divisor
8
9 print("Dividend :", dividend)
10 print("Divisor  :", divisor)
11 print("Quotient  :", quotient)
12 print("Remainder :", remainder)
13
14 print("\nOptimization:")
15 print("Non-Restoring Division avoids repeated restoration steps.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104> & C:\Users\dines\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dines/OneDrive/Documents/GitHub/AI_MLA0104/LAB_MLA0104/mla0104"
Dividend : 13
Divisor  : 3
Quotient  : 4
Remainder : 1

Optimization:
Non-Restoring Division avoids repeated restoration steps.
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104> █
```

5. Optimization

- Replace Restoring Division with Non-Restoring Division.
- Eliminate repeated restoration steps.
- Reduce arithmetic operations.
- Improve execution speed.
- Suitable for embedded and real-time systems.

Feature	Restoring	Non-Restoring
Restoration	Yes	No
Speed	Slow	Fast
Operations	More	Less

6. Testing & Expected Output

Dividend	Divisor	Quotient	Remainder
13	3	4	1
20	5	4	0
15	4	3	3

7. Conclusion

The Restoring Division Algorithm is slower because it repeatedly restores negative partial remainders. The **Non-Restoring Division Algorithm** eliminates these unnecessary operations, reducing execution time and improving processor performance.

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Debugging IEEE 754 Floating-Point Addition

Problem Statement

A scientific application using **IEEE 754 single-precision floating-point representation** produces inaccurate results when adding very small and very large numbers. Analyze the causes of the error and suggest optimization techniques to improve accuracy.

1. Debugging Analysis

IEEE 754 **single precision** uses:

- 1 Sign bit
- 8 Exponent bits
- 23 Mantissa bits

Example:

Large Number = 100000000

Small Number = 1

Expected Result

100000001

Single-precision result may become

100000000

Errors Identified

- Exponent alignment shifts the smaller number.
- Precision is lost during normalization.
- Rounding removes low-order bits.

2. Step-by-Step Solution

Step 1: Align Exponents

The smaller number is shifted to match the larger exponent.

Step 2: Add Mantissas

The mantissas are added after alignment.

Step 3: Normalize

Normalize the result if required.

Step 4: Round

Round the mantissa according to IEEE 754 rules.

3. Pseudo Code

START

Input Two Floating Numbers

Align Exponents

Add Mantissas

Normalize Result

Round Result

Display Output

STOP

4. Python Program

```
mla0104 > ...
1  # Q5: Floating Point Precision
2
3  large = 100000000.0
4  small = 1.0
5
6  result = large + small
7
8  print("Large Number :", large)
9  print("Small Number :", small)
10 print("Result      :", result)
11
12 print("\nOptimization:")
13 print("Use double precision to improve accuracy.")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104> & C:\Users\dines\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/dines/OneDrive/Documents/GitHub/AI_MLA0104/LAB_MLA0104/mla0104"
Large Number : 100000000.0
Small Number : 1.0
Result      : 100000001.0

Optimization:
Use double precision to improve accuracy.
PS C:\Users\dines\OneDrive\Documents\GitHub\AI_MLA0104\LAB_MLA0104>
```

5. Optimization

- Use **double precision (64-bit)**.
- Reduce rounding errors.
- Add smaller numbers first when possible.
- Use stable summation algorithms.

Precision Mantissa Bits Accuracy

Single	23	Moderate
Double	52	High

6. Testing & Expected Output

Number 1 Number 2 Result

100000000	1	100000001
10.5	5.5	16.0
1.25	2.75	4.0

7. Conclusion

Precision loss occurs because of **exponent alignment, normalization, and rounding** in IEEE 754 single precision. Using **double precision** and appropriate numerical techniques improves accuracy and minimizes floating-point errors.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided